

Announcements

- Login, open an IDE, and Folio/browser for class
- Check grades in Folio
- Download binaryIO2.zip, binaryIO3.zip, and binaryData.zip

1

Announcements

- Revel Exercises due
 - Chap 17
- Project 8 due Friday, start of lab
 - See Program Submission requirements in Folio

2

Remaining Schedule

- Course evaluations
- Friday May 2 is last lab
- Mon May 5 is last class meeting
- Final day/time on class schedule/syllabus
- Do **NOT** use Gradescope as a calendar

3

Warm-Up Questions

- What is the difference between binary and text files?
- What are the advantage/disadvantages to binary IO?
- How does Java accomplish binary input/output?

4

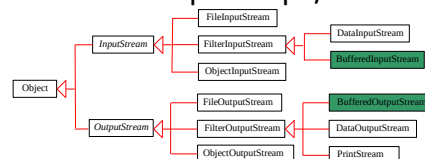
Lecture Questions

- What are the advantages of using buffers?
- How do we use buffers when doing binary I/O?
- How do we know that buffered I/O is beneficial?
- How can we store and retrieve instances of objects in Java?
- How do random access files work?
- How can we implement this idea in Java?
- What considerations need to be made so this works?

5

BufferedInput/OutputStream

- Use buffers to speed up I/O



BufferedInput/OutputStream do not contain any new methods
All the methods in BufferedInput/OutputStream are inherited from the Input/OutputStream classes

6

BufferedInput/OutputStreams

- Can read/write information into a buffer before doing slow task of read/writing to disk
- Once buffer is full, buffer is flushed, which moves data to destination
- Buffer is managed by JVM and automatically reads/writes when full

7

BufferedInput/OutputStreams

```
// Create a BufferedInputStream
public BufferedInputStream(InputStream in)
public BufferedInputStream(InputStream in, int bufferSize)

// Create a BufferedOutputStream
public BufferedOutputStream(OutputStream out)
public BufferedOutputStream(OutputStream out, int bufferSize)
```



8

BufferedInput/OutputStreams

- Can be wrapped around **any** InputStream or OutputStream
 - **NOTE:** Actual type of stream determines methods you can access
- Default buffer size is 512 bytes, but new value can be specified in constructor
- Should generally **always** be used when doing IO, will not see performance in small files, but will see significant gains in large files

9

TestBuffered.java

```
File sourceFile = new File("src/pricelist.dat");
File targetFile = new File("src/processed.dat");
try {
    // Create a buffered input stream
    DataInputStream input = new DataInputStream(
        new BufferedInputStream(
            new FileInputStream(sourceFile)));
    // Create a buffered output stream
    DataOutputStream output = new DataOutputStream(
        new BufferedOutputStream(
            new FileOutputStream(targetFile)));
}
```

10

TestBuffered.java

```
// read from input, write to output
double r, numBytes = 0;
while ((r = input.read()) != -1) {
    output.writeDouble(r * 1.25);
    numBytes++;
}
// Display the copied size
System.out.println(numBytes + " bytes copied");
```

11

Case Studies: Copy File

- Modified to take command line arguments or set default file values

```
java Copy source target
```

- Copies a source file to a target file and displays the number of bytes in the file
 - If source does not exist, tell user file is not found
 - If target file already exists, tell user file already exists



13

Copy.java

```
try {
    BufferedInputStream input = new BufferedInputStream(new FileInputStream(sourceFile));
    BufferedOutputStream output = new BufferedOutputStream(new FileOutputStream(targetFile));
} {
    // Continuously read a byte from input and write it to output
    int r, numberOfBytesCopied = 0;
    while ((r = input.read()) != -1) {
        output.write((byte)r);
        numberOfBytesCopied++;
    }
    System.out.println(sourceFile + " was copied to " + targetFile);
    // Display the file size with count of bytes or length of file
    System.out.println(numberOfBytesCopied + " bytes copied");
    System.out.println(sourceFile.length() + " bytes copied");
}
```

14

Simple Benchmarking

- How can we measure and confirm that buffering is actually faster and/or more efficient, and if so, demonstrably better?
- How can we compare binary vs. text I/O file sizes?
- Turn to **benchmarking** – can simply define as the comparison of results for evaluation

15

Simple Benchmarking

- CAVEAT:** By nature, most benchmarking is prone to inaccuracy
- Java benchmarking is especially fraught with dangers due to the JVM, certain compilers, and many other issues
- All figures and examples are intended to illustrate a general idea of relative performance, look for consistent trends

16

Simple Benchmarking

- For true Java benchmarking, research frameworks like Caliper, JMH, and articles about Java micro-benchmarking
- NetBeans has built-in profiler and eclipse has many plugins
- Lots of variables to be aware of, large topic with lots of details

17

Simple Benchmarking

- Will be using System.nanoTime() which **can be** (and usually is) slightly more accurate than System.currentTimeMillis()
- No guarantees, anti-virus programs, search indexers, other programs, etc. can skew results as well as testing machine

18

Simple Benchmarking

- Compare read/writes using buffered and un-buffered streams
- Compare file sizes of binary vs. text
- Isolate reads vs. writes
- See Timings.java
- Remember caveats, these timings are for a general idea of differences

19

Using Object Streams

- Wrap ObjectInput/OutputStream on any Input/OutputStream using the following constructors:

```
// Create an ObjectInputStream
public ObjectInputStream(InputStream is)

// Create an ObjectOutputStream
public ObjectOutputStream(OutputStream os)
```

26

TestObjectOutputStream.java

```
try ( // output stream for object.dat
    ObjectOutputStream output =
        new ObjectOutputStream(
            new FileOutputStream("object.dat"));
    ) {
    // Write string, double, and object
    output.writeUTF("John");
    output.writeDouble(85.5);
    output.writeObject(new java.util.Date());
}
```

27

TestObjectInputStream.java

```
try ( // Create input stream for object.dat
    ObjectInputStream input = new ObjectInputStream(
        new FileInputStream("object.dat"));
    ) {
    // Read a string, double, and object
    String name = input.readUTF();
    double score = input.readDouble();
    java.util.Date date =
        (java.util.Date)input.readObject();
    System.out.println(name + " " + score + " " + date);
}
```

28

The Serializable Interface

- Not all objects can be written to an output stream
- Objects that can be written to an object stream are said to be **serializable**
- A serializable object is an instance of the **java.io.Serializable** interface
- A serializable object's class must implement Serializable

29

The Serializable Interface

- Serializable is a marker interface
- It has no methods, so no need to add additional code in your class that implements Serializable
- Implementing this interface enables the Java serialization mechanism to automate the process of storing objects and arrays

30

Serializing Arrays

- Array is serializable if all its elements are serializable
- Entire array can be saved into a file using writeObject() and later restored using readObject()
- An example that stores an array of five int values and an array of three strings, and reads them back to display on the console

31

TestObjectStreamForArray.java

```
int[] numbers = {1, 2, 3, 4, 5};
String[] strings = {"John", "Susan", "Kim"};

try ( // Create output stream for array.dat (append)
    ObjectOutputStream output =
        new ObjectOutputStream(
            new FileOutputStream("array.dat", true));
) {
    // Write arrays to the output stream
    output.writeObject(numbers);
    output.writeObject(strings);
}
```

32

TestObjectStreamForArray.java

```
try ( // Create an input stream for file array.dat
    ObjectInputStream input = new ObjectInputStream(
        new FileInputStream("array.dat"));
) {
    int[] newNumbers = (int[])(input.readObject());
    String[] newStrings = (String[])(input.readObject());

    // Display arrays
    for (int i = 0; i < newNumbers.length; i++)
        System.out.print(newNumbers[i] + " ");
    System.out.println();

    for (int i = 0; i < newStrings.length; i++)
        System.out.print(newStrings[i] + " ");
}
```

33

Serializeable Examples

- Imagine a GUI calculator that calculates semester GPA using a Semester class
- Could store a semester, then load stored semester for a student on run
- Expansion: Could store various semesters for a single student and retrieve them as needed, attach dropdown to choose semester

34

Serializeable Examples

```
public void generateStudents() {
    String name = "Jane S. Doe";
    String[] courses = {"CSCI 1302", "MATH 1161", "ENGL 1102",
        "PHYS 212K", "CSCI 3301"};
    int[] credits = {3, 4, 3, 4, 3};
    Semester sem1 = new Semester(name, courses, credits);

    String name2 = "John Q. Doe";
    String[] courses2 = {"PEBC 2000", "MATH 1113",
        "HIST 1111", "ENGL 1101", "CSCI 3301"};
    int[] credits2 = {2, 3, 3, 3, 3};
    Semester sem2 = new Semester(name2, courses2, credits2);

    // Cont.
```

35

Serializeable Examples

```
try ( // Create output stream for object.dat
    ObjectOutputStream output =
        new ObjectOutputStream(
            new BufferedOutputStream(
                new FileOutputStream(
                    "src/students.dat"))));
) {
    // Write objects to the file
    output.writeObject(sem1);
    output.writeObject(sem2);
} catch (Exception e) {
    System.out.println("ERROR");
}
```

36

Serializeable Examples

```
public Semester[] loadStudents() {
    Semester[] arr = new Semester[2];
    try ( // Create an input stream for file object.dat
        ObjectInputStream input = new ObjectInputStream(
            new BufferedInputStream(
                new FileInputStream("src/students.dat"))));
    ) {
        // Read objects from file
        arr[0] = (Semester)(input.readObject());
        arr[1] = (Semester)(input.readObject());
    } catch (Exception e) {
        System.out.println("ERROR");
    }
    return arr;
}
```

37

The transient Keyword

- If an object is **Serializable**, but contains non-serializable instance data fields, can the object be serialized? No
- To enable the object to be serialized, you can use the **transient** keyword to mark non-serializable data fields and the JVM will ignore those fields when writing the object to an object stream (lose that data)

39

The transient Keyword, cont.

- Consider the following class:


```
public class Foo implements java.io.Serializable {
    private int v1;
    private static double v2;
    private transient A v3 = new A();
}
class A { } // A is not serializable
```
- When an object of the Foo class is serialized, only variable v1 is serialized
- Variable v2 is not serialized because it is a static variable, and variable v3 is not serialized because it is marked transient
- If v3 were not marked transient, a java.io.NotSerializableException would occur

40

Random Access Files

- All of the streams you have used so far are known as read-only or write-only streams
- External files of these streams are sequential files that cannot be updated without creating a new file (cannot in-place edit) or appending

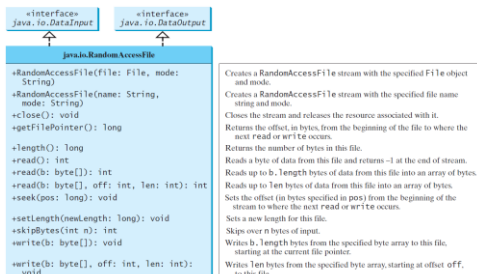
41

Random Access Files

- Often necessary to modify or insert new records into files
- Java provides the **RandomAccessFile** class to allow a file to be read from and written to at random locations

42

RandomAccessFile



43

File Pointer

- Random access file consists of a sequence of bytes
- Special marker called the **file pointer** that is positioned at one of these bytes
- Read or write operations take place at the location of the file pointer

44

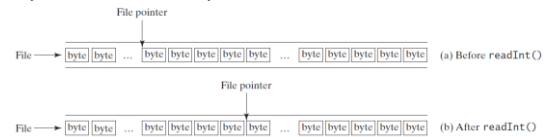
File Pointer

- When a file is opened, the file pointer starts at the beginning of file
- Read or write operations move the file pointer to next data set after performing the read or write

45

File Pointer

- For example, if you read an int value using `readInt()`, the JVM reads four bytes from the file pointer and after the read the file pointer is four bytes ahead of the previous location



46

RandomAccessFile Methods

- Many methods in `RandomAccessFile` are the same as those in `DataInputStream` and `DataOutputStream`
- `readInt()`, `readLong()`, `writeDouble()`, `readLine()`, `writeInt()`, and `writeLong()` can be used in `DataInputStream` or `DataOutputStream` as well as in `RandomAccessFile` streams

47

RandomAccessFile Methods, cont.

- ```
void seek(long pos) throws IOException;
```
- Sets the offset from the beginning of the `RandomAccessFile` stream to where the next read or write occurs
- ```
long getFilePointer() throws IOException;
```
- Returns the current offset, in bytes, from the beginning of the file to where the next read or write occurs

48

RandomAccessFile Methods, cont.

```
long length() throws IOException
```

- Returns the length of the file

```
final void writeChar(int v) throws IOException
```

- Writes a character to the file as a two-byte Unicode, with the high byte written first

```
final void writeChars(String s) throws IOException
```

- Writes a string to the file as a sequence of characters

49

RandomAccessFile Constructors

```
// read and write mode
```

```
RandomAccessFile raf =
```

```
    new RandomAccessFile("test.dat", "rw");
```

```
// read only mode
```

```
RandomAccessFile raf =
```

```
    new RandomAccessFile("test.dat", "r");
```

50

Short Example of RandomAccessFile

- Use seek operation to “jump” to various portions of the file
 - Uses size of data type (int in this case)
 - For instance, an int is 4 bytes, a character is 2 bytes
- Extract and display the value at that location

51

Short Example of RandomAccessFile

- Primitive data type sizes (1 byte = 8 bits):
 - byte: 8-bit, short: 16-bit
 - int: 32-bit, long: 64-bit
 - float: 32-bit
 - double: 64-bit
 - char: 16-bit
- <https://docs.oracle.com/javase/tutorial/ava/nutsandbolts/datatypes.html>

52

TestRandomAccessFile.java

```
try { // Create a random access file
    RandomAccessFile inout = new
        RandomAccessFile("src/inout.dat", "rw");
} {
    // Clear the file to destroy old contents, if any
    inout.setLength(0);

    // Write new integers to the file
    for (int i = 0; i < 200; i++)
        inout.writeInt(i);

    // Display the current length of the file
    System.out.println("Current file length is " + inout.length());
}
```

53

TestRandomAccessFile.java

```
// Retrieve the first number
inout.seek(0); // Move pointer to beginning
System.out.println("The first number is " + inout.readInt());

// Retrieve the second number
inout.seek(1 * 4); // Move pointer to second number
System.out.println("The second number is " + inout.readInt());

// Retrieve the tenth number
inout.seek(9 * 4); // Move pointer to tenth number
System.out.println("The tenth number is " + inout.readInt());
```

54

Case Study: Address Book

- Use RandomAccessFile to create an address book
- Add button stores a new address to the end of the file
- First, Next, Previous, and Last buttons retrieve the first, next, previous, and last address entries from the file, respectively

55

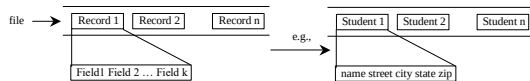
Fixed Length String I/O

- Random access files are often used to process files of records
- For convenience, fixed-length records are used in random access files so that a record can be located easily
- A record consists of a fixed number of fields

56

Fixed Length String I/O

- A field can be a string or a primitive data type
- A string in a fixed-length record has a maximum size
- If a string is smaller than the maximum size, the rest of the string is padded with blanks; if the string is larger, it is truncated to fit the maximum size



57

FixedLengthStringIO.java

```
public static String readFixedLengthString(
    int size, DataInput in) throws IOException {
    // Declare array of characters
    char[] chars = new char[size];
    // Read fixed number of chars
    for (int i = 0; i < size; i++)
        chars[i] = in.readChar();
}
```

58

FixedLengthStringIO.java

```
public static void writeFixedLengthString(String s, int size,
    DataOutput out) throws IOException
{
    char[] chars = new char[size];
    // Fill in string with characters
    s.getChars(0, Math.min(s.length(), size), chars, 0);

    // Fill in blank characters in the rest of the array
    for (int i = Math.min(s.length(), size); i < chars.length; i++)
        chars[i] = ' ';

    // Write a new string padded with blank characters
    out.writeChars(new String(chars));
}
```

59

Address Implementation

Rest of the work can be summarized in the following steps:

1. Create the user interface
2. Add a record to the file
3. Read a record from the file
4. Write the code to implement the button actions

See AddressBook.java

60

Announcements

- Revel Exercises due
 - Chap 17
- Project 8 due Friday, start of lab
 - See Program Submission requirements in Folio

61

Remaining Schedule

- Course evaluations
- Friday May 2 is last lab
- Mon May 5 is last class meeting
- Final day/time on class schedule/syllabus
- Do **NOT** use Gradescope as a calendar

62